

FlowForge

AI-Assisted Distributed Workflow Orchestration & Execution Platform

Project Specification & Build Blueprint

A serious SWE-focused systems project designed to demonstrate backend engineering, distributed systems, workflow orchestration, cloud infrastructure, observability, fault tolerance, and practical AI integration.

Primary Goal	Build a reliable platform that defines, schedules, executes, monitors, and recovers multi-step workflows.
Core Engineering Theme	Distributed workflow execution with dependencies, concurrency, retries, leases, workers, and observability.
AI Theme	AI assists with workflow creation, failure analysis, and optimization - but never becomes a critical dependency.
Target Positioning	Strong software-engineering / backend / distributed-systems portfolio project.

Source of truth: phased FlowForge build plan agreed for the project.

1. What Exactly Is FlowForge?

FlowForge is a platform where a developer can create a workflow made of multiple dependent tasks, run that workflow, and have a set of workers execute those tasks reliably. The workflow is represented as a Directed Acyclic Graph (DAG): each node is a task and each edge represents a dependency.

Example: a machine-learning pipeline may look like **Upload Data -> Validate -> Clean -> Transform -> Train -> Evaluate -> Generate Report**. Independent tasks can execute in parallel, while dependent tasks wait until their prerequisites finish.

The engineering challenge is not simply running functions. FlowForge is about making execution reliable when there are many jobs, multiple workers, retries, timeouts, duplicate messages, and partial failures.

In one sentence: FlowForge is a small, educational version of the kind of workflow-orchestration infrastructure used to run reliable distributed pipelines.

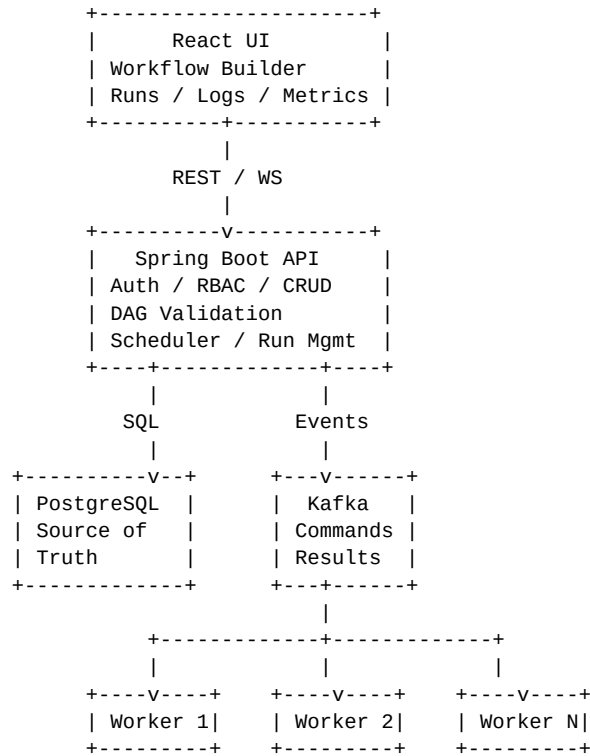
2. Why This Project Is Valuable

- Shows more than CRUD: scheduling, concurrency, dependency management, failure recovery, and distributed execution.
- Demonstrates system-design thinking: source of truth, leases, idempotency, event-driven messaging, observability, and clear service boundaries.
- Adds a backend/distributed-systems project alongside enterprise Spring Boot internship work and the AI/blockchain project.
- Creates measurable engineering evidence through integration tests, failure simulations, load tests, metrics, and architecture decisions.

3. Real-World User Story

Step	What the user does	What FlowForge does
1	Creates a project	Creates a workspace for workflows and executions.
2	Builds a workflow	Stores tasks and dependency edges as a versioned DAG.
3	Triggers a run	Creates a workflow run and identifies tasks that are immediately ready.
4	Execution starts	Scheduler dispatches ready tasks to available workers.
5	Task succeeds	Result is recorded and dependent tasks may become ready.
6	Task fails	Retry policy is applied; after max retries the task can move to dead-letter handling.
7	Worker dies	Lease expires; the task becomes claimable again by another worker.
8	User investigates	Dashboard shows task states, attempts, timing, logs, and failure information.
9	AI assists	AI can generate workflows, explain failures, or suggest optimizations with user approval.

4. High-Level Architecture



Redis -> cache / heartbeats / locks / rate limiting
OTel -> traces
Prometheus -> metrics
Grafana -> dashboards
FastAPI AI Service -> NL workflow / diagnosis / optimization

Important architectural principle: the core execution path must remain deterministic. If the AI service is unavailable, FlowForge must still be able to validate and execute normal workflows.

5. Main Components

Component	Responsibility
React Frontend	Workflow builder, project screens, run history, task states, logs, metrics and AI interactions.
Spring Boot Backend	Authentication, RBAC, workflow CRUD/versioning, DAG validation, scheduling, orchestration and APIs.
Worker Service	Claims tasks, executes work, sends heartbeats/results, handles task attempts and reports failures.
PostgreSQL	Durable source of truth for users, workflows, tasks, dependencies, runs and execution history.
Kafka	Event-driven task commands, results and worker events; supports decoupled distributed execution.
Redis	Fast ephemeral state such as heartbeats, caching, locks and rate limiting where appropriate.

Component	Responsibility
AI Service	FastAPI service for structured workflow generation, failure analysis and optimization suggestions.
Observability Stack	OpenTelemetry, Prometheus and Grafana for traces, metrics and operational visibility.

6. Complete Technology Stack

Layer	Technology	Purpose
Frontend	React + TypeScript	Application UI and typed client logic.
UI Styling	Tailwind CSS	Fast, consistent interface styling.
Workflow Graph	React Flow	Visual DAG builder with nodes and dependency edges.
Charts	Recharts	Execution and performance charts.
Data Fetching	TanStack Query	Server-state caching, fetching and mutations.
Realtime	WebSocket or SSE	Live task/run updates.
Backend	Java + Spring Boot	Core REST API and orchestration logic.
Security	Spring Security + JWT	Authentication and server-side authorization.
Persistence	JPA/Hibernate	Mapping Java entities to PostgreSQL.
Database	PostgreSQL	Transactional source of truth.
Messaging	Apache Kafka	Asynchronous task and event transport.
Cache/Coordination	Redis	Caching, heartbeats, locks and lightweight coordination.
AI	Python + FastAPI + Pydantic	Separate AI service with validated structured outputs.
AI Model	LLM API	Natural-language workflow generation and reasoning assistance.
Containers	Docker + Docker Compose	Repeatable local environment and multi-worker simulation.
CI/CD	GitHub Actions	Build, test, integration checks and Docker builds.
Cloud	GCP	Optional production-style deployment in final phase.
Telemetry	OpenTelemetry	Distributed traces and correlation.
Metrics	Prometheus	Time-series operational metrics.
Dashboards	Grafana	System and worker health visualization.

Layer	Technology	Purpose
Testing	JUnit, Mockito, Testcontainers	Unit + integration testing with real dependencies.
Load Testing	k6 or Gatling	Measured throughput and latency under workload.

7. Repository Structure

```

flowforge/
├── frontend/
│   ├── src/
│   │   ├── pages/
│   │   ├── components/
│   │   ├── workflow/
│   │   ├── api/
│   │   ├── hooks/
│   │   └── package.json
│   └──
├── backend/
│   ├── src/main/java/.../
│   │   ├── auth/
│   │   ├── project/
│   │   ├── workflow/
│   │   ├── dag/
│   │   ├── scheduler/
│   │   ├── execution/
│   │   ├── worker/
│   │   └── common/
│   ├── src/test/
│   └──
├── ai-service/
│   ├── app/
│   │   ├── workflow_generation/
│   │   ├── diagnosis/
│   │   ├── optimization/
│   │   └── schemas/
│   └── tests/
│   └──
├── infra/
│   ├── docker/
│   ├── prometheus/
│   ├── grafana/
│   └── compose/
│   └──
├── docs/
│   ├── architecture/
│   ├── adr/
│   ├── api/
│   └── performance/
│   └──
├── docker-compose.yml
└── README.md

```

Worker boundary decision: start with a clean worker module boundary inside the main backend codebase during the earliest phase, then make the worker separately deployable when Phase 2 introduces real worker processes. This keeps the initial implementation manageable without losing the service boundary.

8. Database Design

PostgreSQL is the durable source of truth. The first schema focuses only on entities required to make the core workflow engine work.

Table	Purpose	Key relationships
users	Application users and credentials/roles.	Users own or participate in projects.
projects	Logical workspace for workflows.	Project -> workflows.
workflows	Stable workflow identity.	Workflow -> many versions.
workflow_versions	Immutable/versioned workflow definition.	Version -> tasks and dependencies.
tasks	Nodes in a workflow graph.	Belongs to a workflow version.
task_dependencies	Directed edges between tasks.	Source task -> dependent task.
workflow_runs	One execution instance of a workflow version.	Run -> many task runs.
task_runs	One task execution attempt lifecycle.	Task run -> attempts/results/state.

Core states

Workflow run: CREATED, RUNNING, SUCCESS, FAILED, CANCELLED. **Task:** PENDING, READY, RUNNING, SUCCESS, FAILED, RETRYING, DEAD_LETTER, CANCELLED.

ER-style relationship

```
User 1---* Project
Project 1---* Workflow
Workflow 1---* WorkflowVersion
WorkflowVersion 1---* Task
Task *---* Task (through TaskDependency)
WorkflowVersion 1---* WorkflowRun
WorkflowRun 1---* TaskRun
Task 1---* TaskRun
```

Important rule: workflow versions should be immutable once executed. A new edit produces a new version so historical runs remain reproducible.

9. OpenAPI / API Contract Sketch

Area	Example endpoints	Purpose
Auth	POST /api/auth/register POST /api/auth/login	Create account and obtain JWT.
Projects	GET/POST /api/projects GET/PATCH/DELETE /api/projects/{id}	Project CRUD with authorization.
Workflows	GET/POST /api/projects/{id}/workflows	Create and list workflow definitions.
Versions	POST /api/workflows/{id}/versions GET /api/versions/{id}	Create and read immutable workflow versions.

Area	Example endpoints	Purpose
Runs	POST /api/versions/{id}/runs GET /api/runs/{id}	Trigger and inspect executions.
Tasks	GET /api/runs/{id}/tasks	Inspect task-level execution state.
Logs	GET /api/task-runs/{id}/logs	Retrieve execution diagnostics.

API design principle: validate ownership/RBAC at the backend boundary. The UI is not trusted for authorization.

10. DAG Engine

The DAG engine is the heart of FlowForge. A workflow is valid only if its dependency graph is acyclic. The engine therefore performs graph construction, validation, cycle detection, topological ordering, and ready-task calculation.

Core algorithms

- Cycle detection: DFS recursion stack or Kahn's algorithm.
- Topological sort: Kahn's algorithm or DFS postorder.
- Ready-task detection: task is ready when all dependency tasks are SUCCESS.
- Dependency unlocking: after a task succeeds, update dependents and identify newly ready nodes.
- Parallelism: independent ready tasks can be submitted to separate worker threads/processes.

Tests must include self-loop, simple cycle, longer cycle, disconnected graph, diamond dependency, single-node workflow, and parallel independent tasks.

11. Scheduling & Distributed Execution

The scheduler decides which tasks are eligible to execute. In the distributed phases, the scheduler dispatches work to separate workers rather than running everything inside the API process.

Mechanism	What it solves
Worker registration	Lets the platform know which worker instances exist.
Heartbeat	Shows that a worker is alive.
Lease	Temporarily reserves a task for one worker.
Lease renewal	Prevents healthy long-running tasks from being reclaimed.
Lease expiry	Allows recovery when a worker disappears.
Retry policy	Handles transient task failures automatically.
Dead-letter state	Stops endlessly failing tasks from blocking the workflow.
Idempotency key	Prevents duplicate delivery from creating duplicate side effects where supported.

The system should use explicit task/run states and state transitions rather than relying on implicit flags scattered through the code.

12. Kafka Event Model

Kafka is introduced after the core worker/recovery path is proven. Suggested topics:

- **task-commands** - scheduler sends executable task instructions.
- **task-results** - workers publish success/failure/results metadata.
- **worker-events** - registration, heartbeat and lifecycle events.
- **dead-letter-events** - terminal failures requiring inspection or manual action.

Kafka gives an event-driven boundary and decouples the API/scheduler from worker availability. Because delivery may be at-least-once, task handling needs idempotency-aware logic.

13. AI Layer

The AI component is deliberately separated from the deterministic execution engine.

Feature	Input	Output	Safety boundary
Natural-language workflow generation	User description	Structured workflow JSON	Pydantic schema + DAG validation + explicit user approval before saving.
Failure analysis	Sanitized logs, task state, retries, timestamps	Human-readable diagnosis and likely causes	No direct mutation of runtime state.
Optimization suggestions	Historical runs, durations, dependency structure	Suggestions for parallelization, bottlenecks, retries	Suggestions require user review.

Non-negotiable principle: AI can advise and generate structured proposals; it must not receive arbitrary shell execution or unrestricted production mutation privileges.

14. Security & Access Control

- JWT-based authentication.
- Password hashing using a secure password encoder rather than storing plaintext passwords.
- RBAC roles: Admin, Developer, Viewer.
- Server-side authorization for every project/workflow/run mutation.
- Input validation for workflow JSON, task parameters, identifiers and version references.
- Audit logs for important mutations and administrative actions.
- AI service receives only the minimum data required for its feature.
- Avoid dynamic shell command execution from untrusted task definitions.

15. Frontend Experience

Screen	What it shows
Login/Register	Authentication.
Projects	Projects available to the user.
Workflow Builder	React Flow canvas, task nodes, connections, validation, save/version.

Screen	What it shows
Run Dashboard	Current workflow state, progress and task statuses.
Task Detail	Attempts, duration, worker, errors, logs and retry history.
Workers	Online/offline workers, heartbeat age, workload and utilization.
Observability	Queue depth, throughput, retry rate and latency charts.
AI Assistant	Generate workflow, explain failure, suggest optimization.

16. Phased Build Plan

The project should be built in layers. The goal is to finish a smaller system completely before adding distributed complexity.

Phase	Main outcome	Key work	Definition of done
0	Design + scaffold	Repo, schema, API sketch, ADR, hello-world.	Frontend and backend boot and communicate.
1	Single-node engine	Auth, CRUD, DAG validation, in-process scheduler, retries, React Flow.	Build DAG -> save -> run -> parallel tasks -> dependencies work.
2	Real workers + recovery	Worker registration, heartbeat, leases, reclaim, retries, SSE/WS.	Kill one of 3 workers and workflow still completes.
3	Docker + tests + CI	Compose, Testcontainers, failure tests, GitHub Actions.	Clean clone can run the system; CI passes.
4	Kafka	Commands/results/events, idempotency, DLQ.	Distributed recovery works through Kafka.
5	Observability + load	OTel, Prometheus, Grafana, k6/Gatling.	Real measured throughput/latency and dashboards.
6	AI layer	FastAPI, workflow generation, diagnosis, optimization.	AI can be turned off without breaking core execution.
7	Cloud + polish	GCP, leader election, docs, performance report.	Production-style deployment and polished documentation.

17. Testing Strategy

Type	Examples
Unit	Cycle detection, topological sort, task readiness, retry backoff, state transitions.
Integration	Spring Boot + PostgreSQL, API auth, workflow persistence, scheduler behavior.
Distributed	Two workers racing for one task; only one lease winner.
Failure simulation	Worker crash, task timeout, repeated failure, lease expiration.
End-to-end	Create workflow in UI -> trigger -> execute -> observe -> inspect result.
Load	Large task counts and multiple workers; measure throughput and P95/P99.

Every performance number used in documentation or a resume bullet must come from an actual reproducible test run.

18. Observability

- Correlation ID from API request through workflow run, task run, Kafka event and worker execution.

- Distributed traces across API -> scheduler -> Kafka -> worker.
- Metrics for queue depth, task throughput, retry rate, execution duration, worker utilization and failure rate.
- Grafana dashboards for workflow health and infrastructure health.
- Logs should be structured and searchable, with secrets excluded.

19. Local Deployment

Docker Compose should run the complete educational environment, including the frontend, backend, PostgreSQL, Redis, Kafka and multiple workers as the phases mature.

```
docker compose up --build
```

Expected developer flow:

1. Open frontend
2. Log in
3. Create project
4. Build workflow
5. Save version
6. Trigger run
7. Observe tasks execute
8. Stop a worker
9. Watch lease expiry / reassignment
10. Inspect logs and metrics

20. CI/CD

GitHub Actions should eventually run: formatting/lint checks -> unit tests -> integration tests -> build -> Docker image build. Deployment to GCP can be added in the final stretch phase.

21. ADR-001: Why DAGs?

Decision: Model workflows as Directed Acyclic Graphs.

Reasoning: DAGs naturally represent task dependencies while allowing independent tasks to run in parallel. Acyclicity makes execution order tractable and avoids infinite dependency loops.

Trade-off: Cyclic workflows are not represented directly. A future iteration could model controlled loops as explicit workflow constructs, but the core engine stays DAG-based.

22. ADR-001: Why PostgreSQL as the Source of Truth?

Decision: PostgreSQL stores durable workflow and execution state.

Reasoning: The platform needs transactional updates, consistent relationships, historical execution records, queryable state and reliable persistence. PostgreSQL provides a strong relational model for this core data.

Role of Redis/Kafka: They support fast coordination and event transport, but the durable business state remains recoverable from PostgreSQL.

23. The Final Demo Story

A strong demo should show the engineering problem instead of only showing screens:

1. Open FlowForge and create a workflow with parallel branches.
2. Run it and show two independent tasks executing concurrently.
3. Show downstream tasks waiting for prerequisites.
4. Open worker/metrics view and show active workers.
5. Kill one worker during a task.
6. Show heartbeat loss and lease expiry.
7. Show the task being reclaimed and the workflow completing.
8. Open tracing/metrics and inspect the run.
9. Ask the AI assistant why a task failed or how the workflow could be optimized.
10. Turn off the AI service and demonstrate that normal workflow execution still works.

24. Resume Positioning

The project should be positioned around measurable engineering outcomes rather than generic statements such as "built a workflow app." Strong evidence includes number of workers tested, task volume, throughput, P95/P99 latency, recovery time after worker failure, test coverage, and the exact reliability mechanisms implemented.

Example resume themes

- Designed and implemented a distributed DAG workflow orchestration platform using Java, Spring Boot, PostgreSQL, Kafka and Docker.
- Built lease-based task execution with worker heartbeats, retries and failure recovery across multiple workers.
- Added OpenTelemetry/Prometheus/Grafana observability and load testing with measured throughput and tail latency.
- Integrated a bounded FastAPI/LLM service for structured workflow generation and failure diagnosis with deterministic backend validation.

25. What NOT to Build Too Early

- Do not start with Kubernetes, microservices everywhere, or leader-election protocols before the single-node engine works.
- Do not let AI drive core scheduling logic.
- Do not build dozens of screens before execution correctness is proven.
- Do not claim performance numbers without measured tests.
- Do not split the codebase into many deployable services before the boundaries are justified by the execution model.

26. Success Criteria

Area	Minimum success condition
Correctness	Valid DAGs run; cyclic graphs are rejected; dependencies are respected.

Area	Minimum success condition
Concurrency	Independent tasks can execute concurrently.
Reliability	Retries and worker-loss recovery work in a repeatable test.
Durability	Workflow and run history survive process restarts.
Distribution	Multiple workers can execute tasks without duplicate ownership of a leased task.
Observability	A user can understand what happened during a run.
AI	AI improves developer experience without becoming a critical dependency.
Reproducibility	A clean clone can start the environment through Docker Compose.
Documentation	Architecture, API, ADRs, setup and measured performance are documented.

Bottom line: FlowForge is not primarily a UI project. Its value is in the execution engine and the engineering around it: DAG correctness, scheduling, concurrency, distributed workers, failure recovery, messaging, persistence, observability, testing and bounded AI assistance.

27. Recommended Build Order

Start with Phase 0: schema + ER diagram -> API contract -> ADR-001 -> repository scaffold -> Spring Boot/React hello-world. Then move to the single-node engine before introducing real workers, Kafka, observability and AI.